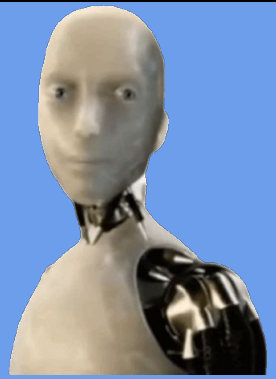
Cloud Platform Deep Dive M2:W4





Cloud AI & MLOps Fundamentals

Over the past three weeks you built the foundations of a cloud environment: virtual machines and IAM, networks, databases, and serverless functions. We use AI to operate and automate those systems faster, and we learn when not to trust it.

By the end of this week you will be able to:

- Explain where AI fits in cloud engineering
- Use cloud AI tools for simple automation tasks
- Write prompts that support cloud operations work
- Describe the MLOps lifecycle at a high level
- Apply responsible AI practices

What is AI and where does it fit in Cloud Engineering?

AI, **Artificial Intelligence**, is technology that enables computers to perform tasks that normally require human intelligence, such as understanding language, recognizing images, making decisions, and solving problems. Generative AI, including large language models like Claude, learns from huge text datasets to produce new content: writing, code, images, and more.

AI is useful for automating repetitive work, analyzing data, and assisting with creative or analytical tasks. However, it is important to know it also has limits: it can make mistakes, reflect biases in its training data, and doesn't "understand" the way humans do.

There are two kinds you'll hear about:

Traditional AI: Looks at data and predicts a label

and Generative AI (GenAI): Creates new content: text, code, images, audio

Places AI helps a cloud engineer

- Operations: log summaries, incident timelines, alert triage
- Automation: generating scripts, IaC, runbooks, config
- Support: ticket classification, drafting responses, knowledge-base search
- Security and cost: spotting odd access patterns, reviewing IAM policies, finding waste

Two ways to use AI

- Assist: you use a chat or console tool to speed up your own work
- Embed: you call an AI API from your pipeline, Lambda, or script (e.g. a function that summarises each failed deployment and posts it to chat)

Gen AI Fundamentals



Generative AI (GenAI) is AI that *creates new content* (text, images, code, audio, video) instead of just analyzing or classifying existing data.

How it differs from traditional AI:

- Traditional AI: “Is this email spam?” (predicts a label)
- GenAI: “Write me an email reply.” (produces something new)

How it works: It’s trained on huge amounts of existing content and learns the patterns in it. Given a prompt, it generates new output that follows those patterns. LLMs (like Claude or GPT) do this for text; other models do it for images (e.g. Midjourney, DALL·E) or audio and video.

Examples:

- Chatbots and writing assistants
- Code generation (GitHub Copilot)
- Image creation from a text description
- Summarising documents or meetings

Key caveat: It generates what *sounds* right, not what is guaranteed true, so it can hallucinate. Always verify important output.

LLMs

Briefly, an **LLM** is a GenAI model trained on huge amounts of text (Claude, GPT, Gemini) with very advanced autocomplete. It has read enormous amounts of text and learned which words tend to follow which. When you give it a prompt, it predicts the next word, then the next, and so on. It is optimized to sound right, not exactly to be right. It makes mistakes, reflects bias in its training data, and doesn’t understand the way humans do

Gen AI Fundamentals (contd.)

Key terms

- **Prompt:** What you type in. Your question or instructions.
- **Token:** A chunk of text, roughly 3/4 of a word. “Cloud computing” is about 2-3 tokens. Models read and bill in tokens, not words.
- **Context window:** The model’s short-term memory. It can only consider this much text (your prompt, the conversation so far, and its reply) at once. Paste in a log that’s too big and it won’t fit or will lose the start.
- **Temperature:** A randomness dial. Low (near 0) gives the most likely, consistent answers, good for configs and code. High gives more varied, creative answers.
- **System prompt:** Hidden standing instructions set by the developer, e.g. “You are a helpful AWS assistant. Keep answers short.” It shapes every reply, and users usually don’t see it.

Strengths and weaknesses: It’s good at language work: summarising, rewriting, classifying, explaining, and drafting code or configs. It’s weak at things that need guaranteed accuracy or information it doesn’t have:

- Exact facts with no source
- Anything after its training cutoff (current events)
- Precise maths (it predicts digits rather than calculating)
- Your private environment (it can’t see your accounts, files, or network unless you give it access)

Hallucination: When the model doesn’t know, it doesn’t say “I don’t know” by default. It generates what a correct answer would look like. In cloud work, that means invented CLI flags, fake API fields, or services that don’t exist. The output reads confidently, so the rule is: verify before you run it (check the official docs or `--help`).

Cost: You pay per token, both input and output. Pasting a huge log or asking for a long answer costs more than a short question.

Cloud AI services

You don’t need to buy GPUs or build ML tools. Cloud providers host AI for you, and you pay as you go. These services come in three layers:

1. Infrastructure: raw computing power (GPUs, TPUs, custom chips) for training and running your own models. Like buying ingredients.
2. Platforms: managed environments to build, train, deploy, and monitor models. Like a fully equipped kitchen.
3. AI APIs and foundation models: ready-made models you call over an API, with little or no ML expertise. Like ordering takeaway.

Most of what we do today happens at the **top layer**.

Layer	**AWS**	**GCP**	**Azure**
Foundation models via API	Bedrock	Vertex AI (Model Garden, Gemini)	Azure OpenAI Service, Azure AI Foundry
Build and train your own ML	Sagemaker	Vertex AI	Azure Machine Learning
Prebuilt AI APIs (vision, speech, text)	Rekognition, Transcribe, Comprehend	Vision AI, Speech-to-Text, Natural Language	Azure AI Vision, Azure AI Speech, Azure AI Language

Lab 1: Log Summariser

Log summariser. Paste this into your playground(Bedrock or Vertex AI Studio). It's a log from a web application.

```
2026-10-27 09:12:03 ERROR web-01 nginx: connect() failed (111: Connection refused) while connecting to
upstream 10.0.2.15:8080
2026-10-27 09:12:04 ERROR web-01 nginx: connect() failed (111: Connection refused) while connecting to
upstream 10.0.2.15:8080
2026-10-27 09:12:10 WARN  app-01 systemd: app.service: Main process exited, code=killed, status=9/KILL
2026-10-27 09:12:10 INFO  app-01 kernel: Out of memory: Killed process 2314 (java)
2026-10-27 09:12:15 INFO  app-01 systemd: app.service: Scheduled restart job
```

Prompt: "You are a cloud support engineer. Summarise this log in 3 bullet points, state the likely root cause, and suggest two next steps."

As one reads the answer, we can check:

- Did it notice that the app was killed because the server ran out of memory?
- Did it explain why nginx then showed "connection refused"?
- Are the next steps sensible and safe to run?

Lab 2: Ticket Classifier, and a Security Rule

Here are two tickets:

> **Ticket #4821:** From Sarah (Finance). *Subject: Can't open the reports bucket.* "Hi, since this morning I get 'Access Denied' when I open the monthly-reports bucket in S3. It worked yesterday. I need the October report for a board meeting at 3 PM today. Please help urgently!"
>
> **Ticket #4822:** From David (Dev team). *Subject: Resize the dev VM.* "When you get a chance, could the dev-test VM be resized to a larger instance type? No rush, anytime this week is fine."

Prompt:

> "You are a cloud support triage assistant. For each ticket, return JSON with: `category` (Access/IAM, Compute, Networking, Billing, Other), `priority` (High, Medium, Low), and `summary` (one sentence). Return JSON only."

Ask the AI for structured output, give it the support tickets and ask for JSON:

```
{ "category": "...", "priority": "...", "summary": "..." }
```

Prompting for Cloud Operations

RCTFC

- **Role:** “You are a senior cloud engineer”
- **Context:** the log, the architecture, the constraints
- **Task:** what exactly to do
- **Format:** bullets, JSON, a table, a bash script
- **Constraints:** length, tools, “ask me if unsure,” “do not invent commands”

Before and after:

- *Weak:* “fix my server”
- *Strong:* “You are a Linux admin. Ubuntu 24.04 VM on AWS, nginx returns 502. Here are the last 20 lines of the error log: [log]. List the three most likely causes, ordered by probability, and give the command to check each one.”

Prompt Practice

1. How would you rewrite “why is my bucket not working” into a strong prompt.
2. Ask for a bash script that warns when disk usage is above 80%.
3. Prompt for a small IAM policy with least-privilege access.

MLOps: DevOps for Machine Learning

Why does MLOps exist? A model that works well today can quietly get worse next month. MLOps is how teams notice that, and fix it.

MLOps applies DevOps ideas (automation, CI/CD, monitoring) to machine learning.

The ML lifecycle is a loop:

Data → Prepare → Train → Evaluate → Register → Deploy → Monitor → Retrain

How it differs from DevOps:

- You version **data and models**, not just code.
- Models **drift**. Real-world data changes, so accuracy drops over time.
- Training must be **reproducible**: the same data and settings should give the same model.
- Testing covers **model quality**, not just “does it run.”

Key terms:

- **Feature:** an input the model learns from
- **Training vs inference:** learning from data vs using the model to make predictions
- **Model registry:** a catalogue of versioned models
- **Pipeline:** an automated chain of lifecycle steps
- **Drift:** a model getting less accurate as the world changes
- **Canary release:** sending a small share of traffic to a new model first

Using AI Responsibly

AI can speed up your work, but it can also make mistakes, expose data, or act on bad instructions. Responsible use means building habits that protect your systems, your company, and your users. Here are five.

1. Protect privacy
Anything you type into an AI tool leaves your machine. Never paste passwords, access keys, customer data, or internal URLs into a prompt. Replace them with placeholders like `<API_KEY>` or `customer-123`. Also find out where your data goes and whether the provider uses it for training. Cloud services like Bedrock and Vertex AI run inside your own account under your own permissions, which is why companies prefer them to public chat tools.

2. Check for accuracy
AI is built to sound right, not to be right. It can invent commands, flags, and services that don't exist, and it will state them confidently. Before you run anything, check it against the official documentation or `--help`, and test it somewhere safe first.

3. Keep a human in the loop
Let AI draft, and let a person approve. This matters most for actions that are hard to undo: deleting resources, changing IAM permissions, opening security groups, or spending money. If a command can cause damage, you read it, understand it, and take responsibility for running it.

4. Watch for bias and unfair outcomes
AI learns from human-written data, so it can pick up human bias. If it's used to filter applications, flag suspicious accounts, or prioritise support tickets, test it with a wide range of cases and check that it doesn't treat groups of people differently without good reason.

5. Be accountable and transparent
Write down where AI is used in your systems, who owns the results, and keep logs of what it was asked and what it did. If something goes wrong, you need to be able to trace it and explain it.

Using AI Responsibly (contd.)

Apply least privilege to AI too.
Last month you learned that people and services should have only the permissions they need. The same applies here: an AI function or agent that summarises logs doesn't need permission to delete buckets.

Think it through:
An AI assistant suggests `aws s3 rm s3://company-logs --recursive` to "free up space." Before running it, what would you check?

Which bucket is this, and what's in it? Do you have a backup? Is versioning on? Does anyone else depend on those logs? Who needs to approve the deletion? Could you do it on a small test first?

Fix a 502 Incident with AI (Lab 3)

The scenario
You're the on-call engineer. A web application on EC2 sits behind a load balancer. After a new release, users are getting **502 Bad Gateway** errors. You'll use an AI tool to investigate and communicate, and you'll practise checking its work.

Use this log:

```
2026-10-28 14:02:11 INFO  deploy:  Release v2.4.1 deployed to app-01
2026-10-28 14:02:14 ERROR app-01 app.service: ModuleNotFoundError: No module named 'psycopg2'
2026-10-28 14:02:14 WARN  app-01 systemd: app.service: Main process exited, code=exited, status=1/FAILURE
2026-10-28 14:02:20 ERROR alb: Target app-01:8080 health check failed (connection refused)
2026-10-28 14:02:25 ERROR alb: 502 Bad Gateway returned to client 203.0.113.25
```

Task 1: Diagnose
Write a prompt using the RCTFC framework (Role, Context, Task, Format, Constraints) that asks the AI to summarise the log and list **three likely causes**, from most to least likely. A good prompt gives the AI the log, says where the app runs, and tells it what format you want. The more specific your context, the more useful the answer.

Task 2: Build a checklist
Ask the AI for a troubleshooting checklist **as a table**, with columns such as Step, Command, and What you expect to see. This turns an answer into something you can follow under pressure.

Task 3: Communicate
Ask for a short incident update for **non-technical stakeholders**: no jargon, what happened, what's being done, and when the next update will come. Engineers have to explain problems to managers and customers, and AI is useful for drafting that.

Task 4: Verify before you trust
Ask the AI for a bash command to check one of the causes. Then **check the command yourself** against `--help` or the documentation. Mark it **Verified** or **Rejected**, and write one line saying why. Don't run anything you don't understand.

Task 5: Reflect (3 sentences)

- One thing the AI got wrong, missed, or oversimplified
- One safeguard you would use at work
- One thing you would never paste into the tool

Quiz & Extra Learning Materials

. Bedrock docs https://docs.aws.amazon.com/bedrock/
· Vertex AI docs https://cloud.google.com/vertex-ai/docs
· Google Cloud Skills Boost https://www.cloudskillsboost.google/ \
· AWS Responsible AI https://aws.amazon.com/ai/responsible-ai/

1. Name two tasks where AI can help a cloud engineer.
2. What is a hallucination, and why does it matter for cloud work?
3. What does MLOps add compared with DevOps?
4. What is model drift?
5. Give two responsible-AI practices for production systems.